

OPTION 1

AI Trading Research Assistant — Mini Prototype

Estimated time: 3–4 hours

Deadline: 3 days

AI tools: Allowed and encouraged

No prior trading expertise required

Assignment Brief

We are exploring an **AI-native trading research platform** where a user can ask questions about markets in natural language.

For example:

“Does buying NIFTY after a 1% fall have an edge?”

The system should eventually be able to:

Understand the question → Structure it as an experiment → Test it → Explain the result → Remember what it learned

For this assignment, you only need to build a **small functional part of this experience**.

Your Task

Build a small web prototype where a user can enter a question such as:

“Does buying NIFTY after a 1% fall work better during high-volatility periods?”

The application should:

1. Understand the question

Identify relevant information such as:

- Instrument
- Timeframe
- Entry condition
- Exit condition
- Holding period
- Variables / filters
- What the user is trying to find out

2. Convert it into a structured experiment

For example:

EXPERIMENT

Instrument: NIFTY

Timeframe: Daily

Entry:

NIFTY falls $\geq 1\%$

Filter:

High volatility

Exit:

Not specified

Holding period:

Not specified

Question:

Does the strategy have a positive edge?

The exact structure is up to you.

3. Identify missing information

The system should recognize when important information has not been provided.

For example:

“How long should the position be held?”

The system should ask the user to clarify rather than blindly making important assumptions.

4. Show the final experiment

Present the structured experiment in a clean and understandable interface.

Technical Expectations

You may use any appropriate technology.

We would like to see a modern web implementation using technologies such as:

- React / Next.js
- TypeScript / JavaScript
- APIs
- LLM APIs
- Any suitable database or storage

You are free to choose your implementation approach.

AI coding tools such as Cursor, Claude, ChatGPT, GitHub Copilot, Gemini, etc. are allowed and encouraged.

However, **you should understand and be able to explain the code you submit.**

What We Will Evaluate

We will look at:

1. **Product thinking** — Did you understand what the user is actually trying to accomplish?
2. **AI implementation** — Did you use AI meaningfully rather than simply creating a chatbot wrapper?
3. **Technical architecture** — Is the solution reasonably structured and extensible?
4. **Handling ambiguity** — Does the system recognize when information is missing?
5. **User experience** — Is the interaction simple and intuitive?
6. **Code quality** — Is the code understandable, organized and maintainable?
7. **Your thinking** — Can you explain why you made your technical decisions?

Important

Do not build a full trading platform.

We are deliberately giving you a small problem from a much larger system.

We are interested in seeing **how you approach the problem, not how much code you can produce.**

You may use AI tools extensively.

Please include a short note describing:

- Which AI tools you used
- What you used them for
- What parts you personally designed
- What parts you reviewed or modified

Submission

Please submit:

1. **Working prototype** — deployed link preferred
2. **GitHub repository**
3. **Short README** explaining:
 - Architecture
 - Technologies used
 - AI tools used
 - Key decisions
 - What you would improve with more time
4. **2–3 minute screen recording** demonstrating the prototype

Bonus — Optional

If you want to go one step further, you can demonstrate how your structured experiment could eventually be passed to a backtesting engine.

This is completely optional and will not be required for evaluation.

OPTION 2

AI Full-Stack Developer Intern — Thinking & Building Challenge

Estimated time: 3–4 hours

Deadline: 3 days

AI tools: Allowed and encouraged

About the Challenge

We are building an **AI-native trading research platform**.

The long-term vision is to build a system that can help users move from:

Question → Hypothesis → Experiment → Evidence → Learning

For this assignment, we want you to build a **small functional part of this experience**.

This is not a conventional coding test.

We are primarily interested in seeing:

How you think, how you approach an ambiguous problem, how you make decisions, and how you solve problems.

The Problem

Imagine a user enters:

“Does buying NIFTY after a sharp fall work?”

The eventual platform should help the user investigate this question systematically.

However, the question is deliberately incomplete.

Your job is to decide **how the system should handle it**.

There is **no single correct solution**.

Part 1 — Think

Before building the prototype, prepare a short **Thinking Note**.

1. How would you interpret the question?

What does:

“sharp fall”

mean to you?

What information would you need before testing the idea?

2. What assumptions would you make?

Where information is missing, what assumptions would you make and why?

Clearly distinguish between:

- What the user actually said
- What you assumed
- What the system should ask the user

3. What would you ask the user?

Identify the **minimum important questions** that need to be answered before the idea can become a meaningful experiment.

4. What would the experiment look like?

Define, in your own way:

- Market / instrument
- Entry condition
- Exit condition
- Holding period
- Test period

- Relevant filters
- Important costs or assumptions

5. What could go wrong?

Think about how the system could produce a misleading or incorrect conclusion.

Consider issues such as:

- Ambiguous definitions
- Incorrect assumptions
- Data quality
- Look-ahead bias
- Transaction costs
- Slippage
- Overfitting
- Insufficient evidence

You do not need to know all the answers.

We want to see whether you identify the right questions and risks.

Part 2 — Build

Build a **small working web prototype** based on your thinking.

The prototype should demonstrate the following journey:

ASK

The user enters:

“Does buying NIFTY after a sharp fall work?”

↓

CLARIFY

The system should identify important missing information.

It may ask the user questions or present reasonable assumptions for confirmation.

The system should **not blindly invent important parameters without making them visible to the user.**

↓

DEFINE

Convert the question into a structured research experiment.

For example:

MARKET

NIFTY

CONDITION

[Your definition]

ENTRY

[Your definition]

EXIT

[Your definition]

HOLDING PERIOD

[Your definition]

TEST PERIOD

[Your definition]

COST ASSUMPTIONS

[Your definition]

HYPOTHESIS

[Your interpretation]

The exact structure and design are up to you.

↓

TEST

Demonstrate how the experiment could be tested.

You may use:

- Sample data
- A publicly available dataset
- Simulated / mock data

You **do not need to build a production-grade backtesting engine**.

The purpose is to demonstrate the research workflow and your thinking.

↓

LEARN

Show the outcome of the experiment in a way that a normal user can understand.

For example:

Result

[Your result]

What the data shows

[Your interpretation]

What we can reasonably conclude

[Your conclusion]

What should we investigate next?

[Your suggested next questions]

Importantly, distinguish between:

What the data actually shows

and

What the system believes or concludes from it.

Part 3 — Your Design

We are deliberately **not providing a detailed UI specification**.

You decide:

- The screens
- User flow
- Interface
- Architecture
- Data structure
- AI interaction
- Technology
- How the experiment is represented

We want to see **your solution, not our solution reproduced.**

AI Usage

AI tools are **allowed and encouraged.**

You may use:

- ChatGPT
- Claude
- Cursor
- GitHub Copilot
- Gemini
- Other AI development tools

However:

Do not simply copy and paste an AI-generated solution.

We want to understand how you use AI as a development partner.

Include a short **AI Usage Note** explaining:

1. Which AI tools did you use?
2. What did you use them for?
3. Which important decisions did you make yourself?
4. Did you reject or modify any AI-generated suggestions? Why?
5. What part of the solution are you most proud of?

There is **no penalty for using AI extensively.**

We are evaluating whether you can **think with AI rather than simply copy from it.**

Part 4 — Explain Your Work

Submit a **2–3 minute screen recording** demonstrating your prototype.

Briefly explain:

- What you built
- Why you designed it this way
- Your key assumptions
- How AI was used
- What you would improve with more time

You should be able to explain and defend the important decisions in your solution.

Submission

Please submit:

1. Working Prototype

A deployed link is preferred.

2. GitHub Repository

Include the complete source code.

3. Thinking Note

Maximum **2 pages**.

4. README

Include:

- Architecture
- Technology choices
- Key assumptions
- How to run the project

- AI tools used
- What you would improve next

5. AI Usage Note

Maximum **1 page**.

6. 2–3 Minute Demo Video

Show the complete user journey.

Evaluation

We will evaluate the submission primarily on:

Area	Weight
Critical Thinking	25%
Original / Independent Thinking	20%
Problem Solving	20%
Product Thinking	15%
Technical Implementation	15%
Communication	5%

We are NOT primarily looking for:

- The most beautiful UI
- The most lines of code
- The most sophisticated AI model
- The largest number of features

We ARE looking for:

- Good questions
- Intelligent assumptions
- Clear reasoning
- Recognition of ambiguity

- Ability to challenge assumptions
 - Practical problem solving
 - A working prototype
 - Understanding of your own implementation
-

Important

There is no “correct” solution.

We may even disagree with your approach.

What matters is whether you can clearly explain:

Why did you make the choices you made?

and whether you can intelligently reconsider those choices when challenged.

Build less. Think more.

We are looking for people who can help us build something that does not yet have a textbook solution.